Real-world — a support-reply router — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python.

- Hint 1 — One reply chain per category, then branch
- Hint 2 — The plan, and how to prove routing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — One reply chain per category, then branch

Basic Version

You already know how to build one reply chain — `prompt | model | parser`. A router is just **several** of those, and a rule that picks one:

- Build one chain per category (billing, technical) and one **default** chain (for everything else). Each is a normal `ChatPromptTemplate.from_messages([("system", "..."), ("human", "{text}")]) | model | StrOutputParser()`.
- `RunnableBranch((condition, chain), (condition, chain), default_chain)` runs the first chain whose condition is true, else the default — an if / elif / else made of chains.
- The condition is a function of the input: `lambda ticket: ticket["category"] == "billing"`.
- Add one last step, `RunnableLambda(tag_reply)`, that appends `"(auto-routed)"` so you can see the pipeline ran end to end.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Keep each part in its own small function so the pipeline reads like a sentence:

- `build_reply_chain(model, system_text) -> Runnable` — one category's chain.
- `build_router(model) -> Runnable` — the `RunnableBranch` over all categories.
- `build_pipeline(model) -> Runnable` — `build_router(model) | RunnableLambda(tag_reply)`.
- The default (last) argument to `RunnableBranch` has **no condition** — it's what runs when nothing matched. Don't forget it, or an unknown category raises.

Difference between Basic and Intermediate: Basic names the pieces. Intermediate puts each in a named builder so the router is testable and reusable — the shape Project 6 builds on.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and how to prove routing

Basic Version

```
billing   = from_messages([system billing,   human {text}]) | model | parser
technical = from_messages([system technical, human {text}]) | model | parser
general   = from_messages([system general,   human {text}]) | model | parser

router = RunnableBranch(
    (lambda t: t["category"] == "billing",   billing),
    (lambda t: t["category"] == "technical", technical),
    general,                 # default — no condition
)
pipeline = router | RunnableLambda(tag_reply)

run one ticket per category, including an unknown one, and print each reply
```

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

To prove routing **without** spending a token, give each branch a tiny fake model that echoes the system line it was handed. Then the reply tells you which branch ran:

```
fake model: return AIMessage(content = the system message it received)
test 1: billing -> reply mentions "billing agent";
        technical -> "tech-support agent"; sales -> "general support agent"
test 2: an unknown category -> the default branch, and "(auto-routed)" is there
```

Because the branch condition is on `ticket["category"]`, not on the model's words, the fake model is enough to prove the routing is correct. Then run `main()` once with a real `ChatOpenAI` to see real replies.

Difference between Basic and Intermediate: Basic runs the router once. Intermediate checks each route with an OK/WRONG `case` line using a fake model — routing proven for free, replies shown for real.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution